

IUT Nantes
Pôle Sciences et technologie

Nantes Université

BUT Info 2

Cryptographie

2025-2026

Exercice 1 Nombres premiers

1.1. Démontrer que si un nombre n n'est pas premier son plus petit diviseur premier est inférieur ou égal à $\sqrt{n}$. Ecrire une fonction permettant de tester si un nombre est premier (on pourra éventuellement utiliser le résultat démontré)

1.2. Ecrire une fonction permettant de renvoyer la liste des nombres premiers inférieurs à un nombre passé en argument. Calculer la proportion de nombre premiers inférieurs à n. Comparer à $\frac{1}{ln(n)}$ pour remarquer que les nombres premiers ont tendance à se raréfier quand on envisage des nombres de plus en plus grands.

1.3. Bonus :

a. Ecrire une fonction permettant de tester (sans stratégie particulière) si les nombres 13, 1009, 10007,...

100000000000031 sont premiers (nombres rappelés dans une liste dans le fichier `tp1_crypto.py`). Consigner dans une liste le temps mis pour obtenir la réponse.

b. Représenter avec matplotlib le nuage de points montrant la relation entre le nombre de chiffres apparaissant dans l'écriture décimale de chacun de ces nombres et le temps mis pour renvoyer le résultat.

c. Remarquer la nature de ce lien si on représente en ordonnées le logarithme du temps de calcul plutôt que le temps de calcul lui-même. Effectuer une régression linéaire avec la bibliothèque scipy.stats entre les deux variables représentées.

Exercice 2

2.1. Déterminer le pgcd de 3315 et 154 et les coefficients de Bézout associés en détaillant vos calculs avec une démarche liée avec celle de l'algorithme d'Euclide.

2.2. Justifier le fait que 154 soit inversible dans $\mathbb{Z}_{/3315\mathbb{Z}}$ et donner son inverse. Tous les éléments non nuls de $\mathbb{Z}_{/3315\mathbb{Z}}$ sont-ils inversibles ? (Justifier)

2.3. Quel est le plus petit entier naturel n tel que $n \times 154 \equiv 1533[3315]$?

2.4. Ecrire une fonction réalisant l'algorithme d'Euclide étendu en prenant en arguments deux entiers naturels a et b et en renvoyant le triplet formé du pgcd d de ces deux entiers et de deux entiers u et v vérifiant $au + bv = d$.

Exercice 3

3.1. Démontrer que le nombre $7^n + 1$ est divisible par 8 si n est impair et donner le reste de sa division par 8 dans le cas où n est pair.

3.2. Trouver le reste de la division par 13 du nombre 100^{1000}.

Exercice 4 chiffrement affine

Un dictionnaire est fourni pour permettre d'associer à chaque lettre une valeur correspondant à sa position dans l'alphabet en plus de certains caractères spéciaux (comme on va travailler dans $\mathbb{Z}_{/31\mathbb{Z}}$ la 31ème position attribué au caractère ' :' est associée à la valeur 0.

4.1. Mettre en place un dictionnaire permettant d'associer à chaque valeur dans $\mathbb{Z}_{/31\mathbb{Z}}$ le caractère qu'elle représente.

4.2. Réaliser une fonction `chiff_affine(mess,a,b)` permettant le chiffrement d'un message par transformation de chaque caractère à partir du calcul dans $\mathbb{Z}_{/31\mathbb{Z}}$ de $a * position(caractere) + b$. La fonction renverra le message chiffré sous forme de texte

4.3. Réaliser une fonction `dechiff_affine(mess,a,b)` permettant de déchiffrer un message reçu

4.4. Si on ignore la clé utilisée, combien de clés va-t-on devoir tester si on souhaite casser ce système en explorant exhaustivement toutes les solutions ?

4.5. Par quel procédé (expliquez-en le principe de manière détaillée) les deux correspondants peuvent-ils échanger secrètement la clé (couple d'entiers) ?

Exercice 5 Problème du logarithme discret

5.1. Ecrire une fonction renvoyant, pour un nombre premier p passé en argument, un générateur de $(Z_{/p\mathbb{Z}})^*$ choisi aléatoirement (choisir un nombre aléatoire et tester s'il est générateur et reproduire l'opération jusqu'à ce que ce soit le cas).

5.2. Ecrire une fonction `log_discret(y,g,p)` pour un élément y de $(Z_{/p\mathbb{Z}})^*$ et un générateur g de $(Z_{/p\mathbb{Z}})^*$, l'élément x tel que $g^x \equiv y[p]$.

5.3. Ecrire une fonction, avec l'algorithme de Shanks des pas de bébé, pas de géant :

- Soit $s = 1 + \lfloor\sqrt{p}\rfloor$
- Calculer g^{-s}
- Créer deux listes :
  - $L_1 : 1,\ g,\ g^2,\ g^3, \cdots, g^{s-1}$
  - $L_2 : y,\ y.g^{-s},,\ y.g^{-2s},,\ y.g^{-3s}, \cdots ,,\ y.g^{-(s-1)\times s}$
- Trouver une occurrence commune g^{r_0} et $y.g^{-k_0 s}$ respectivement dans les listes L_1 et L_2
- $x = r_0 + k_0.s$ est une solution

5.4. Bonus : Justifier que cet algorithme permet bien de renvoyer le logarithme discret de y (s'il existe). Pour cela en posant x_0 (avec $1 \leq x_0 \leq p-1$ ce logarithme discret tel que $g^{x_0} = y$, considérer la division euclidienne de x_0 par $s : x_0 = k_0 \times s + r_0$.

5.5. Bonus : Comparer les temps d'exécution moyens, des deux fonctions réalisées pour l'obtention du log discret, par exemple avec un échantillon de 100 essais avec $p = 10007$ et des générateurs g et des valeurs cibles y choisis « aléatoirement » (la liste des générateurs vous est fournie pour faciliter ces tests).

Exercice 6 RSA

Bernard et Alice communiquent ensemble en utilisant le cryptosystème RSA. Dans une première partie on imagine que Bernard souhaite envoyer un message chiffré (message prenant la forme d'un simple entier). Pour cela il dispose de la clé publique d'Alice (n, e)

6.1. Vous générerez cette clé (comme si vous mettiez une fonction à disposition d'Alice pour qu'elle l'exécute elle-même) avec une fonction `key_creation()` renvoyant un tuple `(n,e,d)` composée de la clé publique `(n,e)` d'Alice et de sa clé privée `d`. Deux nombres premiers p et q distincts seront choisis aléatoirement entre le 26 ème et le 1000 ème (le 26 ème étant égal à 101, ceci assure que le produit sera supérieur à 10000 ce qui est nécessaire si on veut pouvoir crypter et décrypter des nombres de 4 chiffres ou moins).

6.2. Réaliser une fonction `encryption_int(n,e,m)` et une fonction `decryption_int(n,d,c)` permettant respectivement le chiffrement d'un entier m et le déchiffrement d'un entier c. `(n,e)` correspond à la clé publique d'Alice et `d` à sa clé privée

Bernard souhaite maintenant envoyer un message (non confidentiel) signé numériquement.

Pour cela il va envoyer à Alice le message m (que vous imaginerez) ainsi que le chiffrement d'une empreinte de celui-ci générée par une fonction de hachage soit $e(h(m))$ où e représente la fonction `encryption`.

6.3. Réaliser une fonction `encryption(n,e,msg)` qui prend en entrée la clé publique `(n,e)` et un message `msg` correspondant à un nombre entier potentiellement grand, à découper par paquets de 4 chiffres et qui renvoie le message chiffré (qui prendra la forme d'une liste de nombres : voir ci-dessous). Dans le contexte particulier de cette application pour signer et authentifier le message, le nombre `msg` correspondra au hachage obtenu pour le message de Bob (encore appelé empreinte ci-après) et la clé `e` correspondra à la clé privée de Bob

6.4. Réaliser une fonction `decryption(n,d,msg)` qui prend notamment en entrée la clé privée `d` et un message chiffré `msg` et qui renvoie le message décrypté. Dans le contexte particulier de cette application `d` représente l'exposant choisi dans la clé publique de Bernard.

6.5. Pour l'envoi du message de Bernard vous allez :

a. Calculer l'empreinte du message de Bernard en utilisant la bibliothèque `hashlib` :

```
import hashlib as hash
message="Je déclare être bien l'auteur de ce texte par lequel\
je te déclare ma flamme. Bien romantiquement. Bernard"
#Mais Bernard c'est un peu confidentiel ça quand même
message_utf8=message.encode()
'''utilisation de la fonction de hashage SHA256 et encode
en hexadécimal :'''
message_hash=hash.sha256(message).hexdigest()
#conversion en entier base 10 :
number_message_hash=int(message_hash,16)
```

b. Procéder au chiffrement de l'empreinte du message. Quels sont les éléments que Bernard doit envoyer à Alice ?

c. Comment Alice peut-elle s'assurer que Bernard est bien l'auteur de ce message (non répudiation) ? Si cela s'avère bien être le cas on remarquera que l'opération permet aussi de s'assurer de l'intégrité du message reçu par rapport à celui envoyé. Procédez à ces vérifications.

Exercice 7 Code Hamming : application méthode sur papier

On veut pouvoir vérifier l'intégrité d'un message, transmis sous forme de suite de bits, à sa réception. Pour cela sont ajoutés des éléments lors de son envoi selon le code de Hamming (en faisant ici le choix, comme dans le cours, d'indexer la position des bits en commençant à 1 à partir de la gauche).

7.1. Si le message à transmettre est le suivant : 0 1 1 0 1 1 0 0 1 0 . Quel message va être envoyé une fois que des bits de parité auront été rajoutés ? (Détailler la démarche)

7.2. Si on recevait avec l'exploitation du code de Hamming le message suivant (sans rapport avec celui de la question précédente) : 1 0 1 1 0 0 1 0 0 0, que pourrait-on en déduire ?

Exercice 8 Implémentation code Hamming

8.1. On veut pouvoir, à partir d'un message d'information sous forme de liste de bits, renvoyer le message à envoyer intégrant des bits de parité suivant la méthode vue en cours. On se propose ici de le faire avec un message de taille quelconque qui sera codé par un message avec un certain nombre de bits de parité rajoutés (redondance) en fonction de la taille du message initial (à noter qu'on pourrait faire un autre choix consistant à découper le message initial en blocs de taille déterminée pour ensuite appeler la méthode du code de Hamming autant de fois qu'il y a de blocs avec un nombre de bits de parité déterminé selon la taille choisie pour les blocs).

- **a.** Réaliser une fonction `convert_binaire(n)` renvoyant une liste décrivant la décomposition en base 2 d'un nombre `n` (`convert_binaire(14)` renvoyant par exemple `[0,1,1,1]`) et une variante `complet_convert_binaire(n,f)` renvoyant une liste décrivant la décomposition en base 2 d'un nombre `n` en complétant au besoin pour atteindre au moins `f` éléments (`complet_convert_binaire(14,6)` renvoyant par exemple `[0,1,1,1,0,0]`)
- **b.** Réaliser une fonction `det_nb_bit_parite(n)` qui prend en entrée la longueur du message à transmettre et renvoie le nombre de bits de parité à prévoir. L'usage d'une fonction définie par récurrence est recommandé (le cas de base avec une longueur de message égale à 1 devant renvoyer la nécessité d'avoir deux bits de parité)

On peut vérifier que l'application ϕ renvoyant le code c à partir de l'information contenue dans le message à transmettre i est une transformation linéaire (en vérifiant que $\forall i, i',\ \phi(i+i') = \phi(i) + \phi(i')$. Dès lors on va calculer les images de cette application $\phi\colon \mathbb{F}_2^m \to \mathbb{F}_2^n$ en exploitant la matrice M, qu'on appellera matrice génératrice représentant cette application avec $\phi(i) = M \cdot i = c$.

En prenant comme exemple le code de Hamming(7,4) codant des messages de longueur 4 par l'ajout de trois bits de parité, cette matrice génératrice M est la suivante :

$$M = \begin{pmatrix} 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

On peut effectuer le lien avec la précédente présentation du code de Hamming en constatant qu'on retrouve dans les colonnes de la matrice les images respectives des vecteurs $e_1 = (1,0,0,0), e_2 = (0,1,0,0), e_3 = (0,0,1,0)$ et $e_4 = (0,0,0,1)$. En effet en considérant le message `[1,0,0,0]` on peut vérifier que cela conduit au code `[1,1,1,0,0,0,0]` (qu'on retrouve dans la première colonne de la matrice) où les deux premiers éléments égaux à 1 sont les deux premiers bits de parité et le troisième élément (égal à 1 lui-aussi) est le premier bit du message à transmettre.

c. Réaliser une fonction `mat_gen_hamming(mess)` renvoyant la matrice génératrice de Hamming à utiliser pour coder ce message. Cette matrice sera de dimension (n, m) où m désigne la taille du message et $n = m + k$ où k désigne le nombre de bits de parité à utiliser.

8.2. On souhaite maintenant pouvoir, à partir d'un message reçu, contrôler la présence d'une erreur potentielle (en partant de l'hypothèse de la présence d'une seule erreur éventuelle). Dans le cas de la présence d'une erreur on cherchera à renvoyer le message corrigé.

a. Réaliser une fonction intermédiaire `det_nb_bit_parite_recu(mess_recu)` renvoyant le nombre de bits de parité intégrés dans le message reçu.

On peut là encore réaliser les opérations de contrôles prévues sur les bits de parités par l'exploitation d'une matrice (appelée matrice de contrôle) et notée H traduisant l'application linéaire $\psi\colon \mathbb{F}_2^n \to \mathbb{F}_2^k$ renvoyant les sommes de contrôles sur les différents bits de parité. Lorsqu'aucune erreur n'est décelée dans un code c, l'application doit renvoyer le vecteur nul comportant donc k fois la valeur 0. Dans l'exemple du code de Hamming(7,4) présentant 3 bits de parité $p_1,\ p_2$ et p_3 on place en première ligne la somme de contrôle sur p_1, en deuxième ligne la somme de contrôle sur p_2 et en dernière ligne la somme de contrôle sur p_3. Si l'image d'un code par cette fonction est non nul, par exemple $y = (0, 1, 1)$ on pourra retrouver la position du bit incriminé : $pos = \sum_{i=0}^{k} y[i] \times 2^i$ soit le sixième bit pour l'exemple donné.
Ainsi pour ce code de Hamming(7,4) cela donne la matrice de contrôle H suivante :

$$H = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}$$

b. Réaliser une fonction `mat_cont_hamming(mess_recu)` renvoyant la matrice de contrôle de Hamming à utiliser pour contrôler le message reçu. Cette matrice sera de dimension (k, n) où n désigne la taille du message reçu et k le nombre de bits de parité qui y figurent.

Exercice 9

Les fonctions de permutations

Ces fonctions vont permettre de mélanger les bits dans le mot. Par exemple la permutation $P = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 4 & 1 & 3 & 2 \end{pmatrix}$ se comprend comme la transformation plaçant le premier bit en quatrième position, le deuxième en première... Cette opération peut être réalisée par l'exploitation d'une matrice de permutation P :

$$\begin{pmatrix} 2 \\ 4 \\ 3 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix} \times \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}$$

Dans le fichier tpdes.py proposé, les permutations seront exposées comme une liste de valeurs l donnant les positions occupées par les bits avant permutation (synthétique ici mais moins compatible avec une exploitation mathématique). Ainsi avec l'exemple précédent on donnerait la liste : [2, 4, 3, 1].

Créer une fonction `trans_permut_list_to_array(l)` permettant le renvoi de la matrice carrée de permutation de taille correspondante à la liste `l` explicitant la transformation associée à cette permutation selon la description donnée ci-dessus.

Fonction d'expansion

Elle permet de transformer un bloc de 32 bits en un bloc de 48 bits. Elle s'utilise comme les fonctions de permutation mais duplique par ailleurs 16 bits (ceux aux positions 1, 4, 5, 8, 9, 12, 13, 16, 17, 20, 21, 24, 25, 28 et 29) et les dissémine parmi les 48 bits renvoyés. Elle va être utile lors dans la mise en place de la fonction f qui exploite le bloc de droite avec une sous-clé. Voici un extrait de la fonction que l'on utilisera (présentée ici comme une permutation mais notons bien que ça n'en est pas une car elle n'est pas injective) :

Créer une fonction `trans_exp_list_to_array(l,n,p)` permettant le renvoi de la matrice de taille (n, p) associée à la liste `l` réalisant cette transformation par multiplication selon la même idée que celle utilisée pour les permutations. Ici n est supérieur à p mais on pourra réutiliser cette fonction dans le cas contraire d'une compression ultérieurement.

$$E = \begin{pmatrix} 1 & 2 & 3 & 4 & ... & 29 & 30 & 31 & 32 \\ \{2,48\} & 3 & 4 & \{5,7\} & ... & \{42,44\} & 45 & 46 & 47 \end{pmatrix}$$

Vous retrouverez, dans le fichier, la liste de 48 valeurs donnant l'expression intégrale de cette fonction (on pourra la traduire sous forme de matrice de dimensions 48×32).

Méthode de chiffrement

Création de sous-clés La clé K donnée est présentée sur 56 bits, l'ajout de 8 bits de contrôle de parité étant prévus pour s'assurer de son intégrité après la transmission

transmission. Nous utiliserons la clé K suivante :

$$K = \begin{pmatrix} 0 & 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 \end{pmatrix}$$

- On commence par lui faire subir une première permutation $CP1$. Vous devrez pour cela exploiter la fonction `trans_permut_list_to_array(l)` appliquée ici à `list_CP1`. Une fois la permutation réalisée :
- On sépare la clé obtenue en deux parties de 28 bits G (`cleg`) et D (`cled`)

On va alors réaliser le processus suivant pour obtenir 16 sous-clés qu'on pourra stocker dans une matrice de dimensions `(16,48)` :

- Ecraser G et D en effectuant pour chacune un décalage de 1 bit vers la gauche (le premier bit devenant le dernier)
- La clé K_1 est le résultat de la "permutation" par $CP2$ de la concaténation de G et D (attention CP2 est plutôt une compression permutée car elle transforme une clé de 56 bits en clé de 48 bits, 8 d'entre eux étant supprimés)
- Ecraser G et D en effectuant un nouveau décalage de 1 bit vers la gauche
- La clé K_2 est le résultat de la "permutation" par $CP2$ de la concaténation de G et D
- on continue de même jusqu'à la création de K_{16}

Chiffrement du message

Paquetage : On divise le message en paquet de 64 bits en complétant éventuellement les bits manquant par des 0 au début. On chiffre ensuite séparément les différents paquets. On détaille la méthode pour un paquet noté M.

- On commence par lui faire subir une permutation initiale PI : créer pour cela la matrice de permutation de taille 64×64 à partir de `list_PI`.
- On note MG la partie de gauche du résultat (les 32 premiers bits) et MD sa partie droite.

On va ensuite effectuer 16 rondes. Chacune de ces rondes suit le même schéma à ceci près qu'à la ronde k on utilisera la clé K_k. Le processus des rondes suivant est à réaliser dans une `fonction_ronde(Left_block,Right_block,num_ronde)` :

- On applique la fonction d'expansion au bloc MD. On obtient un message $E[MD]$ non plus sur 32 mais sur 48 bits.
- On calcule $E[MD] \oplus K_k$ (lors de la ronde k)
- On découpe ensuite $E[MD] \oplus K_k$ en 8 blocs de 6 bits. Notons $B_1, \cdots, B_8$ ces 8 blocs avec $B_i = x_1x_2x_3x_4x_5x_6$. On considère l'entier n dont l'écriture en binaire est x_1x_6 (donc $0 \leq n \leq 3$). On considère l'entier m dont l'écriture en binaire est $x_2x_3x_4x_5$ (donc $0 \leq m \leq 15$). On va remplacer chaque B_i par l'écriture en binaire du nombre que l'on trouve à la ligne $n+1$ et la colonne $m+1$ de la

matrice de substitution S_i (données dans le fichier `tp_des.py`). Par construction chaque B_i a alors été remplacé par un bloc de 4 bits. En regroupant les 8 blocs de 4 bits on obtient alors $S[E[MD] \oplus K_k]$ en notant S l'application transformant successivement les 8 blocs .

- On applique au résultat précédent la permutation des rondes PR (à définir via une matrice de dimensions 32×32 à partir de `list_PR`) et on obtient ainsi $PR[S[E[MD] \oplus K_k]]$.
- On remplace :

$$MD \text{ par } \underbrace{PR[S[E[MD] \oplus K_k]]}_{f(MD,K_k)} \oplus MG \text{ et } MG \text{ par } MD$$

Une fois les 16 rondes effectuées :

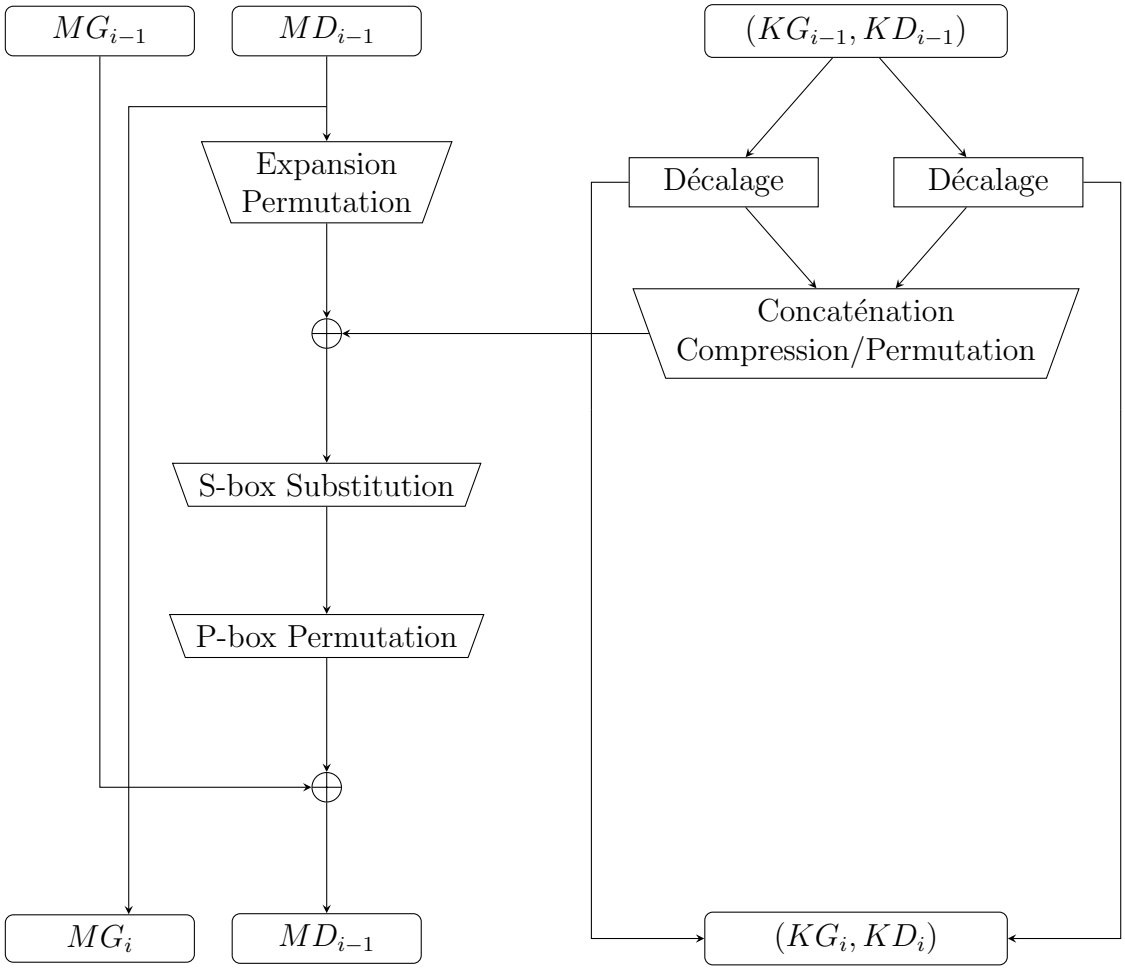
- On recolle la partie gauche et la partie droite obtenue pour obtenir M'
- On applique la permutation inverse de la permutation initiale à M (comment obtient-on PI^{-1} ?) pour obtenir le message chiffré : $C = PI^{-1}[M']$

Déchiffrement du message : Il suffit d'appliquer les mêmes opérations mais dans l'ordre inverse ! En effet on a après chaque ronde $MD_k = f(MD_{k-1}, K_k) \oplus MG_{k-1} = f(MG_k, K_k) \oplus MG_{k-1}$. D'où (comme comme soustraction et addition sont identiques dans $\mathbb{Z}_{/2\mathbb{Z}}$:

$$MG_{k-1} = RG_k \oplus f(MG_k, K_k)$$

Ainsi connaissant MG_k,RG_k et K_k on retrouve MG_{k-1} (et de manière évidente MD_{k-1}. En utilisant à bon escient la fonction

`fonction_ronde(Left_block,Right_block,num_ronde)` définie précédemment vous devez donc pouvoir à moindre frais déchiffrer le message chiffré obtenu.


MG_{i-1}
MD_{i-1}
(KG_{i-1}, KD_{i-1})
Expansion Permutation
Décalage
Décalage
Concaténation Compression/Permutation
S-box Substitution
P-box Permutation
MG_i
MD_{i-1}
(KG_i, KD_i)